SENTRY HOME SECURITY

IoT Backend, ESP32 Firmware & Next.js Admin Dashboard Complete Documentation

1. System Architecture Overview

The Sentry Home Security platform is a hybrid local-mesh and cloud-connected IoT infrastructure designed for real-time perimeter security monitoring. Peripheral sensor nodes (e.g. door contact switches, window sensors, PIR motion detectors) communicate via local **ESP-NOW** low-power protocol to a central ESP32 Gateway node. The Gateway packages telemetry into structured **JSON payloads** and publishes them over **MQTT** to a broker, which feeds the **Express / Node.js backend** and **MongoDB database**. A standalone **Next.js Admin Dashboard** visualizes live telemetry, device topologies, health alerts, and metrics.

Component	Technology / Protocol	Role / Function
Sensor Nodes	ESP32 / ESP-NOW (2.4GHz)	Local low-power sensor sampling & transmission
Central Gateway	ESP32 / WiFi + MQTT	ESP-NOW receiver & MQTT JSON publisher
MQTT Broker	HiveMQ (Topic: sentry-home/telemetry)	Pub/Sub telemetry messaging broker
IoT Backend API	Node.js / Express / Mongoose	Atomic Mongoose upserts, REST API suite
Database	MongoDB ('sentry-home')	Primary Key mapped collections (_id: chipId)
Admin Dashboard	Next.js 14 / Tailwind / Lucide	Real-time UI analytics, alerts & control

2. Telemetry JSON Payload Specification

Every telemetry packet published by the ESP32 Gateway over MQTT to sentry-home/telemetry adheres to a strict, validated JSON schema:

```
{
  "gatewayChipId": "GW_LIVING_ROOM_01",
  "sensorChipId": "SN_FRONT_DOOR_A1",
  "sensorState": "OPEN",
  "battery": 3.75,
  "rssi": -68,
  "gatewayTimestamp": "2026-07-23T23:45:00.000Z"
}
```

Field Name	Data Type	Description & Examples
gatewayChipId	String (Primary Key)	Hardware Chip ID of the receiving router Gateway (e.g. GW_LIVING_ROOM_01)
sensorChipId	String (Primary Key)	Hardware Chip ID of the emitting ESP32 sensor (e.g. SN_FRONT_DOOR_A1)
sensorState	String (Enum)	Current state: OPEN, CLOSED, MOTION_ALERT, or ALARM

battery	Number (Float)	Sampled LiPo battery voltage in Volts (e.g. 3.75V; threshold: 3.3V)
rssi	Number (Integer)	Received Signal Strength Indicator in dBm (e.g. -68 dBm)
gatewayTimestamp	String (ISO 8601)	ISO timestamp recorded upon packet reception at the Gateway node

3. ESP32 Sensor Node Arduino C++ Firmware

The Sensor Node uses **ESP-NOW** to broadcast telemetry directly to the Gateway's MAC address without requiring an access point connection. See full code in docs/esp32_sensor_node.ino.

```
#include
#include

typedef struct __attribute__((packed)) {
  char sensorChipId[24];
  char sensorState[16];
  float battery;
  uint32_t messageId;
} TelemetryPacket;

uint8_t gatewayMacAddress[] = { 0x24, 0x0A, 0xC4, 0x12, 0x34, 0x56 };

void setup() {
  WiFi.mode(WIFI_STA);
  esp_now_init();
  // Read GPIO4 sensor pin & sample ADC GPIO34 battery voltage
  // Send packet via esp_now_send(gatewayMacAddress, ...)
}
```

4. ESP32 Central Gateway Arduino C++ Firmware

The Central Gateway operates in dual AP+STA mode to receive ESP-NOW packets and publish JSON telemetry over WiFi/MQTT. See full code in docs/esp32_gateway.ino.

```
#include
#include
#include
#include

void OnDataRecv(const uint8_t *mac, const uint8_t *data, int len) {
  TelemetryPacket packet;
  memcpy(&packet;, data, sizeof(packet));

  StaticJsonDocument<256> doc;
  doc["gatewayChipId"] = getGatewayChipId();
  doc["sensorChipId"]  = String(packet.sensorChipId);
  doc["sensorState text"] = String(packet.sensorState);
  doc["sensorState"]  = String(packet.sensorState);
  doc["battery"]      = packet.battery;
  doc["rssi"]         = WiFi.RSSI();

  char jsonBuffer[300];
  serializeJson(doc, jsonBuffer);
  mqttClient.publish("sentry-home/telemetry", jsonBuffer);
}
```

5. Backend REST APIs & Admin Dashboard

The Node.js Express backend exposes comprehensive REST API management endpoints under /api/admin:

HTTP Method & Endpoint	Function & Return Value
GET /api/admin/dashboard/summary	Aggregated metrics (Gateways count, Sensor states histogram, low battery count)
GET /api/admin/gateways	Lists all registered central Gateway nodes with connection status and home assignment
GET /api/admin/sensors	Lists all registered ESP-NOW Sensor nodes with battery voltage and RSSI signal
GET /api/admin/dashboard/alerts/low-battery	Filters sensors with battery level below configured threshold (default 3.3V)
GET /api/admin/dashboard/alerts/stale-devices	Inactivity monitor filtering devices with no telemetry for X minutes (1m, 5m, 15m, 30m, 60m)
DELETE /api/admin/devices/:chipId	Removes a specific device and all associated telemetry audit logs from MongoDB
DELETE /api/admin/system/purge-all	Total database purge resetting Gateways, Sensors, and TelemetryLogs
GET / PUT /api/admin/settings	View and update runtime MONGODB_URI, MQTT_BROKER_URL, and MQTT_TELEMETRY_TOPIC